

COMPUTATIONAL LINGUISTIC AND NATURAL LANGUAGE PROCESSING

Class Notes

Unit 1: Introduction to NLP & Classical Approaches

Prepared by

Department of Computer Science & Engineering

Academic Year 2025–26

SECTION 1: WHAT IS NATURAL LANGUAGE PROCESSING?

1.1 Definition and Scope

Natural Language Processing (NLP) is an interdisciplinary field at the intersection of linguistics, computer science, and artificial intelligence. It focuses on enabling computers to understand, interpret, generate, and interact with human language in a meaningful way.

Human languages — such as English, Hindi, or Mandarin — are referred to as **natural languages** to distinguish them from **formal languages** like programming languages (C++, Python). Unlike programming languages, natural languages are ambiguous, context-dependent, and continuously evolving.

 **Core Goal:** To build computational systems that can process, analyse, and generate human language data as effectively as a human would.

Natural vs. formal language — why one is harder

Formal (Python): `x = 5` has exactly one meaning, fixed by the language grammar — there is no other way to read it.

Natural (English): “Time flies like an arrow” has several valid readings — time moves quickly (normal), or “time” the flies that resemble an arrow (a command), or a species “time-flies” enjoys arrows.

A compiler never faces this; an NLP system must resolve such ambiguity on every sentence. That gap is the entire reason the field exists.

1.2 Why is NLP Challenging?

Human language is inherently complex and presents multiple layers of difficulty for machines:

- **Ambiguity** — Words and sentences often carry more than one meaning. Example: “I saw the man with the telescope” (who has the telescope?)
- **Context Dependence** — Meaning changes with context. “Cool” can mean cold, fashionable, or approval.
- **Variability** — Same meaning, different surface forms: “The cat chased the mouse” vs. “The mouse was chased by the cat”
- **World Knowledge** — Requires external knowledge. “He drank the wine though the label said vinegar” requires knowing what wine and vinegar are.
- **Idioms and Metaphor** — “Kick the bucket” does not involve kicking anything.
- **Informal Language** — Abbreviations, slang, and typos in social media text.
- **Low-Resource Languages** — Most NLP research focuses on English; thousands of languages lack labelled data.

More worked examples of ambiguity

Lexical ambiguity: “I went to the bank.” — bank = riverside or financial institution? The single word has two unrelated meanings, and nothing in the sentence settles it.

Structural ambiguity: “Flying planes can be dangerous.” — is the act of flying planes dangerous, or are planes that are flying dangerous? The same words group into two different structures.

Garden-path sentence: “The old man the boats.” — readers first take “man” as a noun and stall; it is actually a verb (to man = to staff), with “the old” as the subject. This shows how we resolve ambiguity incrementally and can be led astray.

Why these matter: each example needs a different kind of knowledge to resolve — word senses for the first, sentence structure for the second, and part-of-speech reasoning for the third. A real NLP system must handle all three, which is why no single trick solves “ambiguity” in general.

1.3 History and Evolution of NLP

Era	Paradigm	Key Features
1950s–1960s	Symbolic / Rule-based	Hand-crafted rules, machine translation (Georgetown experiment, 1954)
1970s–1980s	Structured Knowledge	Semantic networks, conceptual dependency, ELIZA, SHRDLU
1990s	Statistical NLP	Probabilistic models, HMMs, corpus-based approaches
2000s–2010s	Machine Learning	SVM, Maximum Entropy, CRF, feature engineering
2013–2017	Word Embeddings	Word2Vec, GloVe; distributed representations
2018–Present	Deep Learning / LLMs	BERT, GPT, Transformers; pre-train & fine-tune paradigm

What each era looked like in practice

1954 — Georgetown experiment: an early machine-translation demo automatically converted about 60 Russian sentences into English using only 6 grammar rules and a 250-word dictionary — impressive enough to spark huge (and premature) optimism about MT.

1966 — ELIZA: a rule-based “psychotherapist” that worked purely by matching patterns and reflecting the user’s words back, with no real understanding:

User: I am unhappy. ELIZA: Do you think coming here will help you not to be unhappy?

2018 — BERT: learns meaning from context, so it represents “bank” differently in “river bank” and “bank account” — something n-gram models fundamentally cannot do.

1.4 Core Tasks in NLP

NLP encompasses a wide spectrum of tasks at different levels of language understanding:

Low-Level Tasks (Text Processing)

- **Tokenisation** — splitting text into words or subwords
- **Stemming & Lemmatisation** — reducing words to their base forms
- **Part-of-Speech (POS) Tagging** — labelling each word as noun, verb, adjective, etc.
- **Named Entity Recognition (NER)** — identifying names, organisations, locations

Mid-Level Tasks

- **Parsing** — analysing grammatical structure (constituency, dependency)
- **Coreference Resolution** — tracking references to the same entity
- **Word Sense Disambiguation (WSD)** — resolving meaning of ambiguous words

High-Level Tasks (Application-Level)

- **Machine Translation (MT)** — translating between languages
- **Sentiment Analysis** — classifying opinions as positive, negative, neutral
- **Text Summarisation** — generating concise summaries
- **Question Answering (QA)** — producing answers from documents
- **Information Extraction (IE)** — extracting structured facts from text
- **Dialogue Systems / Chatbots** — conversational agents



How tasks chain together — one sentence through a pipeline

Input: “Apple is buying a startup in London.”

1. Tokenise → [Apple] [is] [buying] [a] [startup] [in] [London] [.]
2. POS tag → Apple/NNP is/VBZ buying/VBG a/DT startup/NN in/IN London/NNP
3. NER → Apple = ORG, London = LOC
4. Relation → LOCATED_IN(startup, London)

What this shows: each stage consumes the output of the one before it. The tagger needs tokens; the entity recogniser uses the tags (NNP signals a likely proper-noun entity); and relation extraction uses the entities. A mistake early on — say, splitting “London.” wrongly — would ripple through every later step. This is exactly why the low-level tasks in this section deserve careful attention.

1.5 Applications of NLP

- **Virtual Assistants** — Siri, Alexa, Google Assistant
- **Search Engines** — query understanding and semantic search (Google, Bing)
- **Healthcare** — clinical NLP for EHR analysis, drug extraction
- **Finance** — sentiment analysis of news and social media for stock prediction
- **Customer Service** — automated chatbots and ticket routing
- **Legal Tech** — contract analysis, legal document summarisation
- **Social Media** — hate speech detection, fake news classification
- **Education** — automated essay grading, language learning apps

SECTION 2: LINGUISTIC FOUNDATIONS OF NLP

Before studying computational approaches, it is essential to understand how linguists analyse language. NLP systems must replicate or leverage these levels of linguistic analysis.

2.1 Levels of Linguistic Analysis

Language is not a single, flat phenomenon. Linguists study it as a layered system, where each layer adds a distinct kind of structure on top of the one below it. Sounds combine into words, words combine into sentences, sentences carry meaning, meaning is shaped by context, and context unfolds across an entire conversation. NLP mirrors this organisation: most NLP pipelines are essentially a sequence of modules, each responsible for one level of analysis. Understanding the six levels below tells you both what kind of information exists in language and which NLP task operates on it.

Level	Definition	Typical NLP Task
Phonology	Study of sound systems; how sounds are organised in a language.	Speech recognition, text-to-speech
Morphology	Structure and formation of words (morphemes, affixes).	Stemming, lemmatisation
Syntax	Rules governing sentence structure and word order.	POS tagging, parsing
Semantics	Study of meaning — word, sentence, and truth conditions.	WSD, semantic role labelling
Pragmatics	How context shapes interpretation; speaker intent.	Dialogue, sarcasm/implicature detection
Discourse	How sentences connect into coherent text or conversation.	Coreference, summarisation

1. Phonology — the sound level

Phonology studies how the sounds of a language are organised into meaningful units called **phonemes** — the smallest sound contrasts that change meaning. In English, the difference between /p/ and /b/ distinguishes *pat* from *bat*, so these are separate phonemes. Phonology matters most when language enters or leaves a system as audio.

 **Example:** The words “two”, “to”, and “too” are homophones — identical in sound (/tu:/) but different in spelling and meaning. A speech recogniser must use higher levels (syntax, semantics) to decide which one was intended.

2. Morphology — the word-formation level

Morphology examines the internal structure of words and how they are built from **morphemes** (the smallest meaningful units). Recognising that *unhappiness* is *un-* + *happy* + *-ness* lets a system relate it to *happy* and *unhappy*, which is essential for search, indexing, and dealing with rare word forms. Section 2.2 develops this level in detail.

 **Example:** A search for “run” should also return documents containing runs, running, and ran. Morphological analysis collapses these surface forms to the shared root run.

3. Syntax — the structure level

Syntax governs how words combine into grammatical sentences. It captures **word order** and the hierarchical grouping of words into phrases. Syntax explains why “The dog bit the man” and “The man bit the dog” mean different things even though they use the same words, and why “Man the bit dog the” is not a sentence at all. Section 2.3 covers constituency and dependency structure.

 **Example:** The phrase “old men and women” is syntactically ambiguous: it can group as [old men] and [women], or as old [men and women]. The two groupings give two different meanings.

4. Semantics — the meaning level

Semantics studies literal meaning — the meaning of individual words (**lexical semantics**) and how word meanings combine into the meaning of a whole sentence (**compositional semantics**). It also deals with truth conditions: under what circumstances a sentence is true. A word like *bank* has multiple senses, and choosing the right one is a semantic task (Word Sense Disambiguation).

 **Example:** “Every student read a book” is semantically ambiguous between (a) there is one specific book that all students read, and (b) each student read a possibly different book — a difference in quantifier scope, not word order.

5. Pragmatics — the contextual-meaning level

Pragmatics goes beyond literal meaning to study what a speaker *intends* in a given context. The same sentence can perform different acts depending on who says it, to whom, and why. Pragmatics is responsible for handling indirect requests, politeness, sarcasm, and implicature (meaning that is suggested but not stated).

 **Example:** “Can you pass the salt?” is literally a yes/no question about ability, but pragmatically it is a polite request for action. A useful assistant must respond to the intended request, not answer “yes” and do nothing.

6. Discourse — the multi-sentence level

Discourse analysis looks beyond the individual sentence to how sentences link together into a coherent text or conversation. It covers **coreference** (tracking which entity a pronoun refers to), discourse connectives (*therefore, however*), and the overall flow of information across a document. This level is critical for summarisation, question answering, and dialogue.

 **Example:** “The trophy did not fit in the suitcase because it was too big.” Discourse and world knowledge are needed to resolve it to the trophy rather than the suitcase — the famous Winograd schema problem.

One sentence through all six levels

“The bank can guarantee deposits will eventually cover tuition costs.”

Phonology: disambiguating spoken homophones such as guarantee vs. its pronunciation variants.

Morphology: deposits = deposit + -s (plural); costs = cost + -s.

Syntax: deciding that “deposits” is the object of “guarantee”, not a new subject.

Semantics: choosing the financial-institution sense of bank, not the river-edge sense.

Pragmatics: is this a reassurance to a worried customer, or a formal commitment?

Discourse: linking “tuition costs” to a student mentioned in an earlier sentence.

Key takeaway: A complete NLP system must address all six levels. Classical pipelines handled them as separate, hand-built modules (a tokeniser, then a POS tagger, then a parser, and so on). Modern neural models tend to learn many of these levels jointly from data, but the levels remain a useful map of what language understanding requires.

2.2 Morphology in Detail

Morphology deals with how words are internally structured. A **morpheme** is the smallest meaningful unit of language.

- **Free Morphemes** — can stand alone: dog, run, happy
- **Bound Morphemes** — must attach to another morpheme: -ing, -ness, un-

 **Example:** “unhappiness” = un- (prefix, negation) + happy (root) + -ness (suffix, nominalisation)

Types of morphological processes:

- **Inflection** — adding grammatical information without changing word class: walk → walked, walks
- **Derivation** — creating new words, often changing class: happy (adj) → happiness (noun)
- **Compounding** — combining two or more roots: blackboard, bookshelf

Breaking words into morphemes

cats = cat (root) + -s (plural) → 2 morphemes
unbreakable = un- (not) + break (root) + -able (capable of) → 3 morphemes
nationalise = nation (root) + -al (adj) + -ise (verb) → 3 morphemes
hospitals = hospital (root) + -s (plural) → 2 morphemes

How to read these: start from the root — the part that carries the core meaning — then peel off each affix and name what it does. In “nationalise,” “-al” turns the noun *nation* into the adjective *national*, and “-ise” then turns that into the verb *nationalise*. Each affix changes either the meaning or the word class, which is precisely the information a stemmer or lemmatiser is trying to recover.

Inflection vs. derivation — the key contrast. Inflection keeps the same word, just in a different grammatical form; derivation makes a brand-new word.

Process	Example	What changed
Inflection	teach → teaches, taught, teaching	Tense/agreement only; still the verb 'teach'
Inflection	child → children	Number only; still the noun 'child'
Derivation	teach → teacher	New word, new class (verb → noun)
Derivation	happy → happiness → unhappiness	New words built by stacking affixes

Why it matters for NLP: a lemmatiser should map all the inflected forms of “teach” back to the single lemma *teach*, but it should **not** collapse the derived word *teacher* into *teach* — they are genuinely different words with different meanings.

2.3 Syntax in Detail

Syntax defines which sequences of words form grammatically valid sentences in a language.

Constituency (Phrase Structure)

Sentences are organised into nested hierarchical phrases. Example:

 **Sentence:** [S [NP The quick brown fox] [VP [V chased] [NP the lazy dog]]]

Common Phrase Types: NP (Noun Phrase), VP (Verb Phrase), PP (Prepositional Phrase), AP (Adjective Phrase), ADVP (Adverb Phrase)

Reading a constituency tree as an indented outline

```
S                                     (whole sentence)
├─ NP   The quick brown fox   (subject noun phrase)
└─ VP                                     (predicate)
    ├─ V   chased
    └─ NP   the lazy dog       (object noun phrase)
```

How to read it: the indentation shows containment. The whole sentence (S) splits into a subject NP and a VP; the VP then splits into the verb and its object NP. Each group is a **constituent** — a chunk that behaves as a unit. A quick test: you can replace “the lazy dog” with the single word “it” and the sentence still works, which confirms it is one constituent.

Dependency Structure

Words are linked by directed grammatical relations to a head word, without phrase-level nodes.

 **Example:** In “She gave him a book”: gave is the root; She is nsubj; him is iobj; book is dobj

Reading the relations: the main verb *gave* is the root — everything hangs off it. *She* is its subject (nsubj), *book* is the thing given (direct object, dobj), and *him* is the recipient (indirect object, iobj). This “who-did-what-to-whom” view is why dependency parses are so handy for information extraction.

Structural ambiguity — PP attachment

“She saw the man with the telescope.”

Reading 1: with the telescope attaches to saw → she used a telescope to see him.

Reading 2: with the telescope attaches to the man → the man was holding a telescope.

Why this is the heart of parsing: both readings use the very same words in the same order — the only difference is *where* the prepositional phrase “with the telescope” attaches in the tree. Choosing between them needs context or world knowledge, and (as Section 3.2.3 shows) a probabilistic grammar can pick the more likely attachment automatically.

SECTION 3: CLASSICAL APPROACHES TO NLP

Classical (or traditional) NLP approaches preceded the era of neural networks. They rely on linguistic knowledge, formal rules, and statistical models derived from corpus analysis. Understanding them is critical because:

- They still power many production NLP systems
- They are computationally efficient and highly interpretable
- They form the theoretical backbone upon which modern systems are built

3.1 Rule-Based / Symbolic Approaches

3.1.1 Finite State Automata (FSA) and Transducers

A Finite State Automaton is a mathematical model consisting of states and transitions triggered by input symbols. FSAs are used to model regular languages and are foundational in NLP for:

- **Tokenisation** — recognising word boundaries
- **Lexical analysis** — identifying valid word forms
- **Morphological analysis** — modelling inflections and derivations

 **Key Concept:** A Finite State Transducer (FST) extends FSA by associating an output with each transition. FSTs map input strings to output strings — e.g., mapping surface forms to base forms (“running” → “run”).

Example FSA for recognising the plural of English nouns: the automaton reads a noun and accepts if it ends in ‘s’ (simplified).

A tiny FSA, traced step by step

Goal: accept words made of one or more a’s followed by a single b (ab, aab, aaab...).

States: q0 (start), q1, q2 (accept)

q0 --a--> q1 q1 --a--> q1 q1 --b--> q2

Tracing “aab”: q0 →(a) q1 →(a) q1 →(b) q2. q2 is an accept state → ACCEPTED.

Tracing “ba”: q0 has no transition on ‘b’ → REJECTED.

How to read it: the machine starts in q0 and follows one arrow per input symbol. If it ends in an accept state with no input left, the string belongs to the language; otherwise it is rejected. This is exactly how a tokeniser decides whether a run of characters forms a valid token.

An FST adds an output — stripping a plural

Input surface form: c a t s
Output base form: c a t ϵ (ϵ = empty: the -s is deleted)
Result: the transducer maps “cats” $\rightarrow$ “cat” while emitting the feature +Plural.

Why FSTs are powerful: a plain FSA only says yes/no, but an FST emits something as it reads. Here it reads the surface word and simultaneously writes the lemma plus a grammatical tag. The same machine, run backwards, can *generate* “cats” from “cat +Plural” — which is why FSTs are the classical engine for both morphological analysis and generation.

3.1.2 Regular Expressions

Regular expressions (RegEx) are patterns used to match, search, and manipulate text. They correspond to regular languages and can be implemented as FSAs.

Pattern	Meaning	Example Match
<code>[A-Z][a-z]+</code>	Capitalised word	London, Paris
<code>\d{4}</code>	4-digit number	2024, 1999
<code>\b\w+ing\b</code>	Word ending in -ing	running, walking
<code>(Mr Mrs Dr)\.\s[A-Z]</code>	Titles + name	Dr. Smith

Applications: email extraction, phone number detection, date parsing, NER bootstrapping.

Building a regex piece by piece — matching an email

Pattern: `[\w.]+ @ [\w.]+ \. \w+`
`[\w.]+` $\rightarrow$ one or more letters/digits/dots (the part before @)
`@` $\rightarrow$ a literal at-sign
`[\w.]+` $\rightarrow$ the domain name (e.g. gmail)
`\. \w+` $\rightarrow$ a dot then the suffix (e.g. .com)
Matches: priya.sharma@college.edu Rejects: just-some-text

How to read a regex: scan it left to right as a recipe. Each piece consumes part of the string; “+” means “one or more,” a backslash makes a special character literal (so “\.” is a real dot, not “any character”). Because regexes compile to FSAs, this pattern is just a compact way of writing the state machine that scans for emails.

3.1.3 Context-Free Grammars (CFG)

CFGs use rewrite rules of the form $A \rightarrow \alpha$ (a non-terminal rewrites to a string of terminals and/or non-terminals). They can describe hierarchical sentence structure beyond what regular expressions allow.

Example CFG Rules:

```

S → NP VP
NP → Det N | Det Adj N | Pronoun
VP → V NP | V NP PP
Det → 'the' | 'a'
N → 'dog' | 'cat' | 'bone'
V → 'chased' | 'ate'

```

CFGs are used in:

- Constituency parsing
- Defining syntax of programming languages
- Grammar checking

Deriving a sentence from the grammar, rule by rule

Target: “the dog chased a cat”

S	→ NP VP	(start rule)
NP VP	→ Det N VP	(expand NP)
Det N VP	→ the dog VP	(choose Det→the, N→dog)
the dog VP	→ the dog V NP	(expand VP)
...	→ the dog chased a cat	(fill in V and the object NP)

What a derivation shows: we begin with the start symbol *S* and repeatedly replace a non-terminal using one grammar rule, until only actual words (terminals) remain. Each line applies exactly one rule. If some sequence of rules can produce the sentence, the sentence is grammatical; the record of which rules were used *is* the parse tree. Parsing is just this process run in reverse — starting from the words and discovering which rules must have generated them.

 **Limitation:** CFGs cannot capture all syntactic dependencies in natural language (e.g., agreement across long distances). Extended formalisms like Tree-Adjoining Grammars (TAG) and Head-driven Phrase Structure Grammar (HPSG) address some of these gaps.

3.1.4 Parsing Algorithms

Given a CFG and an input sentence, a parser determines the grammatical structure. Key algorithms:

- **CYK (Cocke-Younger-Kasami)** — dynamic programming algorithm; requires grammar in Chomsky Normal Form (CNF). Time complexity: $O(n^3 \cdot |G|)$.
- **Earley Parser** — top-down, left-to-right parser; handles all CFGs without CNF conversion. More flexible than CYK.

- **Chart Parsing** — uses a table (chart) to store partial parse results, avoiding redundant computation.

 **Ambiguity Problem:** A sentence can have multiple valid parse trees (structural ambiguity).
Example: “I shot an elephant in my pyjamas” — two different attachment sites for the PP.

3.2 Statistical Approaches

Statistical NLP emerged in the 1990s as large text corpora became available. Instead of relying on hand-crafted rules, these approaches **estimate probabilities from data** and then use those probabilities to make predictions. The shift was profound: a rule either fires or it does not, but a probability lets the system rank competing analyses and gracefully prefer the most likely one. This makes statistical systems far more robust to the messy, unseen input that breaks rule-based systems. The four families below — n-gram models, HMMs, PCFGs, and log-linear models — are the workhorses of the classical statistical era.

3.2.1 N-gram Language Models

A **language model** assigns a probability $P(W)$ to a sequence of words $W = w_1, w_2, \dots, w_n$. Intuitively, it answers “how likely is this sentence?” — which is exactly what is needed to rank candidate translations, correct spelling, or predict the next word. By the chain rule of probability:

 **Chain rule:** $P(w_1 \dots w_n) = P(w_1) \cdot P(w_2 | w_1) \cdot P(w_3 | w_1, w_2) \cdot \dots \cdot P(w_n | w_1 \dots w_{n-1})$

Each later factor conditions on the entire history, which is impossible to estimate reliably — most long histories are never seen even in a huge corpus. The **Markov assumption** rescues us: we assume each word depends only on the previous $N-1$ words. This is the defining idea of an n-gram model.

Model	Approximation	Conditions on
Unigram (N=1)	$P(w_i)$	nothing — words independent
Bigram (N=2)	$P(w_i w_{i-1})$	previous word
Trigram (N=3)	$P(w_i w_{i-2}, w_{i-1})$	previous two words

Probabilities are estimated from a training corpus using **Maximum Likelihood Estimation (MLE)** — simply counting and dividing:

 **MLE (bigram):** $P(w_i | w_{i-1}) = \text{Count}(w_{i-1}, w_i) / \text{Count}(w_{i-1})$

Worked example — estimating bigram probabilities

Suppose our entire training corpus is these three sentences, where <s> and </s> mark sentence start and end:

<s> I am Sam </s>

<s> Sam I am </s>

<s> I do not like green eggs and ham </s>

We count how often each word follows another, then divide. For example, the word “I” occurs 3 times; it is followed by “am” twice and “do” once. Some resulting bigram probabilities:

Bigram	Count(w_{i-1}, w_i) / Count(w_{i-1})	Probability
P(I <s>)	2 / 3	0.67
P(Sam <s>)	1 / 3	0.33
P(am I)	2 / 3	0.67
P(do I)	1 / 3	0.33
P(Sam am)	1 / 2	0.50
P(</s> Sam)	1 / 2	0.50

To score a new sentence we multiply the relevant bigram probabilities. For “<s> I am Sam </s>”:

 **Scoring:** $P = P(I | <s>) \cdot P(am | I) \cdot P(Sam | am) \cdot P(</s> | Sam) = 0.67 \cdot 0.67 \cdot 0.50 \cdot 0.50 \approx 0.112$

Bigram vs. trigram — why more context helps

Predicting the next word after “...I want to go to the...”:

Bigram uses only “the” → $P(? | the)$ – huge, vague set: the dog, the idea, the...

Trigram uses “to the” → $P(? | to the)$ – narrower: to the store, to the gym...

The trade-off: a trigram conditions on two previous words instead of one, so its predictions are sharper and usually more accurate. But there is a cost — there are far more possible three-word sequences than two-word ones, so many trigrams are never seen in training (data sparsity), making the counts unreliable. This tension between *more context* and *enough data* is the central practical issue in n-gram modelling, and it is what smoothing tries to manage.

The zero-probability problem and smoothing

MLE assigns probability **zero** to any n-gram it never saw in training. In the corpus above, $P(Sam | green) = 0$, so any test sentence containing “green Sam” would receive probability 0 — and one zero anywhere

wipes out the whole product. Real text is full of valid combinations the corpus happens to miss, so this is fatal. **Smoothing** fixes it by shaving probability mass off seen n-grams and redistributing it to unseen ones.

Technique	Idea
Laplace (Add-One)	Add 1 to every count: $P(w c) = (\text{Count}(c,w)+1) / (\text{Count}(c)+V)$, where V = vocabulary size.
Add-k	Add a fractional k (e.g. 0.05), tuned on held-out data; gentler than add-one.
Interpolation	Mix bigram, unigram, etc.: weighted average of higher- and lower-order models.
Kneser-Ney	State-of-the-art; weights words by how many different contexts they appear in (continuation probability).

Worked Laplace example: in our corpus the unigram vocabulary (including $\langle s \rangle$, $\langle /s \rangle$) has $V = 11$ word types. The unseen bigram “Sam am” would normally be 0, but with add-one smoothing:

 **Add-one:** $P_{\text{lap}}(\text{am} \mid \text{Sam}) = (\text{Count}(\text{Sam}, \text{am})+1) / (\text{Count}(\text{Sam})+V) = (0+1) / (2+11)$
 $= 1/13 \approx 0.077$

So a previously impossible bigram now has a small non-zero probability. The cost is that smoothing slightly under-estimates frequent bigrams — add-one in particular over-corrects, which is why add-k and Kneser-Ney are preferred in practice.

Evaluating a language model — perplexity

We judge a language model by how well it predicts unseen text. The standard intrinsic metric is **perplexity**, the inverse probability of the test set normalised by the number of words. A lower perplexity means the model is less “surprised” by the test data and is therefore better.

 **Perplexity:** $PP(W) = P(w_1 \dots w_n)^{-1/N}$

Interpretation: a perplexity of 100 means the model is, on average, as uncertain as if it had to choose uniformly among 100 words at each step. Reducing perplexity from 200 to 100 roughly halves that uncertainty.

3.2.2 Hidden Markov Models (HMM)

Many NLP problems involve labelling a **sequence**: assigning a POS tag to each word, or a named-entity label to each token. A Hidden Markov Model is the classic probabilistic tool for this. The key idea is that

we observe a sequence of words (the *observations*) but the labels we actually want (the *hidden states*, e.g. POS tags) are not directly visible — we must infer them.

An HMM is defined by five components:

Component	Meaning
States Q	The hidden labels, e.g. POS tags {NN, VB, DT, ...}.
Observations O	The visible symbols, e.g. the actual words.
Transition matrix A	$A[i][j] = P(q_j \mid q_i)$ – probability of moving from state i to state j .
Emission matrix B	$B[i][k] = P(o_k \mid q_i)$ – probability that state i produces word k .
Initial probs π	$\pi[i] = P(q_1 = q_i)$ – probability a sequence starts in state i .

Two assumptions make HMMs tractable: (1) the next state depends only on the current state (Markov assumption on tags), and (2) each word depends only on its own tag (output independence). Three classic algorithms answer the three questions one can ask of an HMM:

Algorithm	Problem solved	Method
Forward	Likelihood: $P(O \mid \lambda)$	Dynamic programming; sums over all paths efficiently.
Viterbi	Decoding: best state sequence	DP; finds the single most likely tag sequence.
Baum-Welch	Learning: estimate A, B, π	Expectation-Maximisation; trains from unlabelled data.

Worked example — Viterbi decoding for POS tagging

Suppose we have a two-tag HMM with states NN (noun) and VB (verb), and we want to tag the sentence “fish sleep”. The model parameters are:



Given parameters

Initial: $\pi(\text{NN})=0.6$ $\pi(\text{VB})=0.4$

Transition: $P(\text{NN}|\text{NN})=0.3$ $P(\text{VB}|\text{NN})=0.7$ $P(\text{NN}|\text{VB})=0.6$ $P(\text{VB}|\text{VB})=0.4$

Emission: $P(\text{fish}|\text{NN})=0.5$ $P(\text{fish}|\text{VB})=0.2$ $P(\text{sleep}|\text{NN})=0.1$

$P(\text{sleep}|\text{VB})=0.6$

Step 1 — first word “fish”. The Viterbi value of each state is the initial probability times the emission:

$$v_1(\text{NN}) = \pi(\text{NN}) \cdot P(\text{fish}|\text{NN}) = 0.6 \times 0.5 = 0.30 \quad v_1(\text{VB}) = 0.4 \times 0.2 = 0.08$$

Step 2 — second word “sleep”. For each state we take the best incoming path (max over previous states) times the emission:

$$v_2(\text{NN}) = \max[0.30 \times 0.3, 0.08 \times 0.6] \times 0.1 = \max[0.090, 0.048] \times 0.1 = 0.0090 \text{ (from NN)}$$

$$v_2(\text{VB}) = \max[0.30 \times 0.7, 0.08 \times 0.4] \times 0.6 = \max[0.210, 0.032] \times 0.6 = 0.1260 \text{ (from NN)}$$

Step 3 — backtrack. The larger final value is $v_2(\text{VB}) = 0.126$, and its back-pointer leads to NN at step 1. The most likely tag sequence is therefore:

 **Result:** fish/NN sleep/VB

This matches our intuition: “fish” is the noun subject and “sleep” is the verb. Crucially, Viterbi found this by global optimisation over the whole sentence, not by tagging each word independently — the transition probabilities pushed it toward the grammatical noun-verb pattern.

3.2.3 Probabilistic Context-Free Grammars (PCFG)

A plain CFG can tell us *whether* a sentence is grammatical and what its possible parse trees are, but when a sentence is ambiguous it cannot say which tree is *more likely*. A PCFG fixes this by attaching a probability to every rewrite rule, with the rule probabilities for a given left-hand side summing to 1. These are estimated from a hand-parsed treebank such as the Penn Treebank.

 **Rule & tree probability:** $P(A \rightarrow \alpha) = \text{Count}(A \rightarrow \alpha) / \text{Count}(A)$ $P(\text{tree } T) = \prod P(r_i) \text{ for every rule } r_i \text{ used in } T$

Worked example: consider the ambiguous sentence “saw the man with the telescope”. With a small PCFG, the two readings correspond to two trees — one where the telescope is attached to the verb phrase (seeing with a telescope) and one where it attaches to the noun phrase (the man who has a telescope). Suppose the relevant rules and their probabilities give:

Reading	Product of rule probabilities	P(tree)
VP-attach (saw with telescope)	$1.0 \times 0.7 \times 0.6 \times \dots$	0.0008
NP-attach (man with telescope)	$1.0 \times 0.3 \times 0.6 \times \dots$	0.0003

Because the VP-attachment tree has the higher probability ($0.0008 > 0.0003$), the parser selects it as the preferred analysis. The probabilistic CYK algorithm computes exactly this — it is the ordinary CYK chart-

parsing algorithm, but in each cell it keeps, for every non-terminal, only the **highest-probability** sub-tree, multiplying child probabilities by the rule probability as it combines them.

⚠ **Limitation:** Standard PCFGs assume rule probabilities are independent of context. In reality, the likelihood of a rule often depends on the specific words involved. Lexicalised PCFGs (Collins, 1997; Charniak, 1997) condition rules on head words to capture this.

3.2.4 Log-Linear / Maximum Entropy Models

HMMs and PCFGs are **generative** — they model how the data is produced — and they impose strong independence assumptions that make it hard to use rich, overlapping clues. Maximum Entropy (MaxEnt), also called a **log-linear** model, is **discriminative**: it directly models the probability of a label y given the input x , and it lets you throw in as many arbitrary, correlated features as you like. The model has the form:

📐 **Formula:** $P(y \mid x) = (1/Z(x)) \cdot \exp(\sum_i \lambda_i f_i(x, y))$

Here each $f_i(x, y)$ is a **feature function** (usually returning 0 or 1), each λ_i is a learned weight, and $Z(x)$ is a normalising constant that makes the probabilities over all labels sum to 1.

Worked example: tagging the word “running”. Useful feature functions for the candidate tag VBG (gerund/present participle) might be:

- $f_1 = 1$ if the word ends in “-ing” and $y = \text{VBG}$ (else 0)
- $f_2 = 1$ if the previous word is “is/was/are” and $y = \text{VBG}$ (else 0)
- $f_3 = 1$ if the word is capitalised and $y = \text{NNP}$ (else 0)

If the learned weights are $\lambda_1 = 1.8$ and $\lambda_2 = 1.2$, then for “*she is running*” both f_1 and f_2 fire for VBG, giving a high score $\exp(1.8 + 1.2)$ before normalisation, so VBG wins easily. Notice that f_1 and f_2 overlap (both relate to the same decision) — something an HMM cannot do cleanly, but MaxEnt handles naturally because it makes no independence assumption among features.

Why “maximum entropy”? Among all probability distributions consistent with the observed feature statistics, the model chooses the one with the highest entropy — i.e. the most uniform, least-committed distribution. This is the most honest choice: it assumes nothing beyond what the features actually tell us. MaxEnt models were the dominant classifiers for POS tagging, parsing, and NER just before neural networks took over.

SECTION 4: CLASSICAL NLP TASKS IN DEPTH

This section examines the core tasks of a classical NLP pipeline. Each task takes the output of the previous one and adds a new layer of structure: raw text is split into tokens, tokens are normalised and reduced to base forms, each token is tagged with its part of speech, the tagged sequence is parsed into

a grammatical structure, ambiguous words are sense-disambiguated, and finally structured facts are extracted. We work through each task with its main methods and concrete examples.

4.1 Tokenisation and Text Normalisation

Tokenisation is the first step in nearly all NLP pipelines. It segments raw character strings into **tokens** — the units (words, punctuation marks, or subwords) that every later stage operates on. It sounds trivial — “split on spaces” — but real text is full of cases where that simple rule fails, and a tokenisation mistake propagates errors through the entire pipeline.

Why tokenisation is hard

- **Contractions** — should “don’t” be one token, or split into “do” + “n’t”? Most parsers prefer the split because “n’t” is a separate negation.
- **Hyphenated words** — is “state-of-the-art” one token or three?
- **Multi-word expressions** — “New York” and “ice cream” denote single concepts but contain spaces.
- **URLs, hashtags, emails** — “#NLP” and “https://example.com” must not be split on their punctuation.
- **Non-whitespace scripts** — Chinese and Japanese write without spaces, so segmentation itself is a hard problem (word segmentation).

 **Example:** “Dr. Smith isn’t in—he’ll be back at 3 p.m.” →

[Dr. | Smith | is | n't | in | – | he | 'll | be | back | at | 3 | p.m.]

Note that “Dr.” and “p.m.” keep their periods (abbreviations), while “isn’t” and “he’ll” are split at the contraction boundary. Deciding this reliably is what makes tokenisation non-trivial.

Text normalisation

After tokenising, text is usually normalised so that superficially different forms are treated alike:

- **Case folding** — lowercasing everything: “Apple” → “apple”. (Risky for NER, where capitalisation signals proper nouns.)
- **Punctuation handling** — remove or keep, depending on the downstream task.
- **Stop-word removal** — dropping very frequent function words (the, is, at) that carry little topical meaning.
- **Spell correction** — using edit-distance algorithms (Levenshtein) to fix typos: “teh” → “the”.

 **Caution:** Normalisation is lossy. Lowercasing helps a topic classifier but hurts NER; stop-word removal helps search but destroys meaning for sentiment (“not good” becomes “good”). Always choose normalisation to fit the task.

4.2 Stemming and Lemmatisation

Both stemming and lemmatisation reduce inflected words to a common base so that *organise*, *organised*, and *organising* are treated as the same item. They differ in how carefully they do it.

Stemming is a fast, crude, rule-based chopping of suffixes. The classic Porter stemmer applies ordered rewrite rules such as “-sses → -ss”, “-ies → -i”, “-ing → ∅”. It does not consult a dictionary, so the output (a **stem**) need not be a real word.

Lemmatisation uses a dictionary and morphological analysis to return the true base form (the **lemma**). It needs to know the word’s part of speech, which makes it slower but accurate.

Property	Stemming	Lemmatisation
Method	Heuristic suffix removal	Dictionary + morphological analysis
Output	Stem (may not be a real word)	Dictionary base form (lemma)
Speed	Very fast	Slower (needs a lexicon)
Needs POS?	No	Yes
Example	studies → studi	studies → study
Tools	Porter, Lovins, Snowball	WordNet Lemmatiser, spaCy

 **Same words, different output:** running → stem: run lemma: run studies → stem: studi lemma: study

better → stem: better lemma: good (lemmatisation knows 'better' is the comparative of 'good')

The “better → good” case shows the real advantage of lemmatisation: it can map irregular forms that no suffix rule could catch. Use stemming when speed matters and the output is only read by a machine (e.g. search indexing); use lemmatisation when you need linguistically correct base forms.

4.3 Part-of-Speech (POS) Tagging

POS tagging assigns a grammatical category — noun, verb, adjective, and so on — to each token. It is a prerequisite for parsing, lemmatisation, and many other tasks. The standard label set for English is the **Penn Treebank tagset** of 36 tags, including NN (singular noun), NNS (plural noun), VB (base verb), VBD (past-tense verb), VBZ (3rd-person singular verb), JJ (adjective), RB (adverb), DT (determiner), and IN (preposition).

 **Example tagging:** The/DT quick/JJ brown/JJ fox/NN jumps/VBZ over/IN
the/DT lazy/JJ dog/NN

The central difficulty is **ambiguity**: the same word can take different tags in different contexts. “book” is a noun in “read a book” (NN) but a verb in “book a flight” (VB); “back” can be a noun, verb, adjective, or adverb. A good tagger resolves this using the surrounding context.

One word, four tags — context decides

the back/NN of the room	– noun (a thing)
back/VB the proposal	– verb (to support)
the back/JJ door	– adjective (which door)
step back/RB	– adverb (direction)

How the tagger decides: it looks at the neighbours. After “the” a noun or adjective is likely; before another noun (“back door”) an adjective fits; a bare verb-like use with no determiner suggests a verb. This is precisely the local context that an HMM captures through transition probabilities $P(\text{tag} \mid \text{previous tag})$ — the model learns, for instance, that DT is very often followed by JJ or NN, rarely by VB.

Classical approaches, in increasing sophistication:

- **Rule-based (Brill tagger)** — assign each word its most frequent tag, then apply transformation rules learned from data to fix errors (“change NN to VB if the previous tag is a modal”). This error-driven learning is interpretable and surprisingly accurate.
- **HMM taggers** — model $P(\text{tags} \mid \text{words}) \propto P(\text{words} \mid \text{tags}) \cdot P(\text{tags})$ and decode with Viterbi (see Section 3.2.2). Fast and effective.
- **Maximum Entropy taggers** — use rich, overlapping features (suffixes, capitalisation, surrounding words) without independence assumptions; among the most accurate classical taggers.

 **Evaluation:** Accuracy = correctly tagged tokens / total tokens. Classical state-of-the-art $\approx 97\%$ on newswire.

That 97% sounds high, but a sentence of 20 words then has roughly a $0.97^{20} \approx 54\%$ chance of being completely correct, which is why even small per-token gains matter for downstream parsing.

4.4 Syntactic Parsing

Parsing assigns a grammatical structure to a sentence, revealing how words relate to one another. There are two dominant paradigms, which answer slightly different questions.

Constituency Parsing

Constituency parsing builds a **phrase-structure tree**: the leaves are words, and internal nodes are phrase labels (NP, VP, PP, S). It shows how words group into nested phrases. Algorithms include CYK with a PCFG and the Earley parser.

 **Constituency tree:** [S [NP The cat] [VP [V chased] [NP the mouse]]]

This says the sentence S is made of a noun phrase “The cat” and a verb phrase “chased the mouse”, and the verb phrase in turn contains the verb plus its object noun phrase.

Dependency Parsing

Dependency parsing instead produces a **graph of directed, typed relations** between words. There are no phrase nodes; every word (except the root) depends on exactly one head word, with a label naming the relation (nsubj = subject, dobj = direct object, det = determiner, amod = adjectival modifier). Dependency structure is often more convenient for extracting “who did what to whom”.

 **Dependency parse of “The cat chases the mouse”**

chases → cat	(nsubj)	cat → The	(det)
chases → mouse	(dobj)	mouse → the	(det)
root:	chases		

Classical dependency-parsing algorithms fall into two families:

- **Graph-based parsing** — treat parsing as finding the maximum spanning tree over all possible word-to-word arcs (McDonald et al., 2005). It searches the whole space exactly, giving high accuracy at higher cost.
- **Transition-based parsing** — build the tree incrementally with a stack and a buffer, using arc-eager or arc-standard actions (Nivre, 2003). A greedy classifier chooses each action, giving very fast linear-time $O(n)$ parsing.

 **Which paradigm?** Constituency trees suit tasks that care about phrase boundaries (e.g. some grammar checkers); dependency trees suit relation/information extraction and are the standard in most modern multilingual pipelines.

4.5 Word Sense Disambiguation (WSD)

Many words are **polysemous** — they have several distinct senses. WSD is the task of choosing the correct sense of a word in context, usually from an inventory defined in a lexical resource such as **WordNet**. It is a semantic task and a long-standing hard problem.

 **Example:** “bank” = (1) financial institution vs. (2) river bank

“The bank offered a low interest rate” → sense (1), financial institution.

The words “interest rate” in the context point clearly to the financial sense — and exploiting such context clues is exactly what WSD algorithms do.

Classical WSD approaches:

- **Lesk algorithm** — a knowledge-based method: for each candidate sense, measure the word overlap between that sense’s dictionary definition (gloss) and the words in the surrounding context; pick the sense with the most overlap. Simple, needs no training data, and a solid baseline.
- **Thesaurus / co-occurrence methods** — use statistics of which words tend to occur together in large corpora.
- **Supervised WSD** — train a classifier (Naive Bayes, SVM) on sense-annotated examples, using context words and features as input. Most accurate, but needs labelled data.

Lesk worked example — disambiguating “pine cone”

Context word: cone. Senses of “pine”:

pine¹ gloss: 'kind of evergreen tree with needle-shaped leaves'

pine² gloss: 'to waste away through sorrow or longing'

The gloss of pine¹ shares words (tree, evergreen) with the gloss of “cone” → choose pine¹ (the tree).

 **Knowledge Acquisition Bottleneck:** Supervised WSD needs sense-tagged corpora, which are expensive and slow to build by hand. This scarcity caps how well supervised methods generalise across domains — a recurring theme in classical NLP.

4.6 Information Extraction (IE)

Information Extraction turns unstructured text into **structured data** — entities, relations, and events that can be stored in a database. Its two foundational sub-tasks are Named Entity Recognition and Relation Extraction.

Named Entity Recognition (NER)

NER finds and classifies proper-name mentions into categories such as PER (person), ORG (organisation), LOC (location), DATE, and MONEY.

 **Example:** [Barack Obama]**PER** was born in [Hawaii]**LOC** in [1961]**DATE**

NER is usually framed as **sequence labelling** using the **IOB scheme** (Inside–Outside–Beginning), where each token gets a tag marking whether it begins an entity (B-), continues one (I-), or lies outside any entity (O):

IOB tagging

Barack/B-PER Obama/I-PER visited/O New/B-LOC York/I-LOC yesterday/O

This token-level framing means any sequence model can do NER. Classical approaches were:

- **Rule-based systems** — gazetteers (lists of known names) plus contextual patterns (“Mr. ____” signals a person).
- **HMM / MEMM** — probabilistic sequence labelling over the IOB tags.
- **Conditional Random Fields (CRF)** — a globally normalised sequence model that scores the whole tag sequence at once, letting it enforce consistency (e.g. an I-PER cannot follow a B-LOC). CRFs were the state of the art for NER and consistently beat HMMs and MEMMs.

Relation Extraction

Relation extraction identifies the semantic relationship that holds between two entities once they have been found.

 **Example:** [Steve Jobs]**PER** founded [Apple]**ORG** → FOUNDED(Steve Jobs, Apple)

Classical approaches include **pattern-based extraction** (e.g. Hearst patterns such as “X such as Y” to find is-a/hyponymy relations), **kernel-based classifiers** over parse-tree structure, and **distantly supervised learning**, which uses an existing knowledge base to automatically label training sentences and so avoids costly manual annotation.

 **Hearst patterns — learning “is-a” facts from raw text**

Pattern: “NP such as NP, NP, ... and NP”

“fruits such as apples, oranges, and mangoes”

→ is-a(apple, fruit) is-a(orange, fruit) is-a(mango, fruit)

Why this works: certain phrasings almost always signal a category relationship. Whenever the words “such as” sit between a general term and a list, the listed items are nearly always examples of that general term. By scanning a huge corpus for this one pattern, a system can harvest thousands of is-a facts automatically — no labelled data required. The weakness is *recall*: the pattern only fires when the text happens to use this exact construction, so many true facts stated differently are missed. This precision-over-recall behaviour is typical of rule-based extraction.

SECTION 5: COMPARISON, LIMITATIONS & TRANSITION TO ML

5.1 Rule-Based vs. Statistical: A Comparison

Criterion	Rule-Based	Statistical
Data needed	None (hand-crafted rules)	Large annotated corpora
Interpretability	High — rules are explicit	Low — black-box distributions
Portability	Low — domain-specific rules	Higher with sufficient data

Criterion	Rule-Based	Statistical
Robustness	Brittle to unseen input	Graceful degradation
Development	Slow (expert effort)	Faster once data available
Performance	High precision, low recall	Good balance with enough data

Same input, two approaches — a sentiment example

Tweet: “omg this phone is sooo gooooood 🥰”

Rule-based: matches words against a hand-built positive/negative lexicon. “sooo” and “gooooood” are not in the lexicon (wrong spelling), and the emoji is not handled — so it may return neutral or fail.

Statistical: trained on millions of real tweets, it has seen these stretched spellings and the 🥰 emoji alongside positive examples, so it confidently predicts positive.

This single example captures the whole table: the rule system is precise but brittle, while the data-driven system degrades gracefully on messy, unseen input.

5.2 Limitations of Classical Approaches

Rule-Based Limitations

- **Knowledge bottleneck** — creating rules is expensive and requires deep linguistic expertise
- **Coverage** — rules cannot exhaustively cover all language phenomena
- **Maintenance** — rules must be manually updated as language evolves
- **No generalisation** — cannot handle unseen patterns gracefully

Statistical Limitations

- **Data sparsity** — n-gram models suffer from unseen n-grams; smoothing is an imperfect fix
- **Lack of long-range dependencies** — bigram/trigram models cannot capture dependencies across long spans
- **Feature engineering** — classical ML models need hand-designed features, requiring domain expertise
- **Bag-of-words assumption** — many models ignore word order and syntactic structure
- **Domain adaptation** — models trained on one domain (news) perform poorly on another (social media)

Seeing the limitations in action

Rule-based brittleness: a rule “tag a word as a date if it matches DD/MM/YYYY” silently misses “Jan 3rd”, “yesterday”, and “03-01-25” — each new format needs a new hand-written rule.

Data sparsity: if the bigram “quantum entanglement” never appeared in training, an n-gram model gives it probability 0, even though it is a perfectly normal phrase; smoothing only patches this crudely.

Long-range dependency: in “The keys to the cabinet ... are on the table”, a trigram model only sees “cabinet ... are” and may wrongly prefer “is” — it cannot reach back to the plural subject “keys”.

Domain shift: a POS tagger trained on newswire that reads “ship it asap, thx” mislabels “ship” (verb here, often a noun in news) and chokes on “asap/thx”, which never appeared in formal text.

5.3 The Road to Modern NLP

The limitations of classical approaches motivated successive paradigm shifts:

- **2001 — Neural Language Models (Bengio et al.)** — first use of feedforward neural networks for language modelling; introduced continuous word representations
- **2008 — NLP from Scratch (Collobert & Weston)** — end-to-end neural models replacing pipeline systems across tasks
- **2013 — Word2Vec (Mikolov et al.)** — efficient dense word embeddings; semantic similarity captured geometrically
- **2014 — GloVe (Pennington et al.)** — global log-bilinear model combining global statistics with local context
- **2015 — LSTMs & Seq2Seq** — recurrent networks for sequence-to-sequence tasks; enabled neural machine translation
- **2017 — Attention & Transformers** — “Attention is All You Need” (Vaswani et al.); parallelisable architecture replacing recurrence
- **2018 — BERT / GPT** — large-scale pre-training on unlabelled text; transfer learning transformed NLP

 **Key Insight:** Classical NLP contributed rigorous mathematical formalisms, evaluation benchmarks, and annotated datasets (Penn Treebank, WordNet, PropBank) that remain foundational resources for modern deep-learning systems.

SECTION 6: SUMMARY, KEY TERMS & EXAM PREPARATION

6.1 Chapter Summary

This unit introduced the field of Natural Language Processing, emphasising:

- NLP is inherently multidisciplinary (linguistics, CS, AI, statistics)
- Language presents profound challenges: ambiguity, context, variability, world knowledge

- NLP has evolved through rule-based, statistical, and neural paradigms
- Classical approaches include FSAs, CFGs, n-gram models, HMMs, and PCFGs
- Each task (tokenisation, POS tagging, parsing, WSD, NER) has classical solutions
- Statistical models improved over rule-based but face sparsity and feature bottlenecks
- Modern deep learning builds on classical foundations and datasets

6.2 Key Terms Glossary

Term	Definition
Tokenisation	Segmenting raw text into tokens (words, punctuation, subwords)
Lemma	The dictionary base form of a word (e.g. 'go' is the lemma of 'went')
POS Tag	Grammatical label assigned to a word token (NN, VB, DT, ...)
N-gram	A contiguous sequence of N words; used in language models
Perplexity	Evaluation metric for language models; lower = better fit to test data
HMM	Probabilistic model of observable outputs generated from hidden states
Viterbi Algorithm	DP algorithm for the most likely hidden-state sequence in an HMM
CFG	Context-Free Grammar; production rules generating syntactic structures
PCFG	Probabilistic CFG; assigns probabilities to grammar rules from corpus counts
CYK Algorithm	DP parsing algorithm for CFGs in Chomsky Normal Form
Constituency Parse	Hierarchical phrase-structure tree for a sentence
Dependency Parse	Head-dependent graph of grammatical relations between words
WSD	Word Sense Disambiguation; selecting the correct word meaning in context
NER	Named Entity Recognition; labelling person, org, location, etc. mentions
IOB Tagging	Inside–Outside–Beginning scheme for labelling token spans
CRF	Conditional Random Field; globally normalised sequence model for tagging
MaxEnt	Maximum Entropy model; log-linear classifier with arbitrary features
Smoothing	Redistributing probability mass to handle unseen n-grams
Lesk Algorithm	WSD using overlap between dictionary definitions and context

6.3 Probable Exam Questions

Short Answer Questions

1. Define NLP. List any five key challenges in processing natural language.
2. What is an n-gram language model? Explain the bigram model with an example.
3. Differentiate between stemming and lemmatisation with suitable examples.
4. What is a Finite State Transducer? How is it used in morphological analysis?
5. Explain the Lesk algorithm for Word Sense Disambiguation.
6. What is the IOB tagging scheme? Illustrate with a NER example.

Long Answer / Descriptive Questions

1. Explain Hidden Markov Models for POS tagging. Describe the three key algorithms (Forward, Viterbi, Baum-Welch) and their roles.
2. What is a PCFG? How does it address structural ambiguity in parsing? Explain the probabilistic CYK algorithm.
3. Compare rule-based and statistical approaches to NLP on interpretability, data requirements, robustness, and scalability.
4. Trace the evolution of NLP from symbolic systems to modern Transformer-based models. Highlight key milestones.
5. Explain Conditional Random Fields (CRF) for Named Entity Recognition. How do CRFs overcome the limitations of HMMs?

Numerical / Problem-Based Questions

1. Given a small corpus, estimate bigram probabilities using MLE. Apply Laplace smoothing to compute the probability of an unseen bigram.
2. Given an HMM with specified transition and emission probabilities, apply the Viterbi algorithm to find the best POS tag sequence for a sentence.
3. Given a CFG, draw the parse tree(s) for an ambiguous sentence. If rule probabilities are given, identify the most probable parse using a PCFG.

6.4 Recommended Reading

- **Jurafsky, D. & Martin, J. H. (2024).** *Speech and Language Processing* (3rd ed. draft). Stanford University. [Primary textbook — Chapters 1–13 cover this unit]
- **Manning, C. D. & Schütze, H. (1999).** *Foundations of Statistical Natural Language Processing*. MIT Press.
- **Bird, S., Klein, E. & Loper, E. (2009).** *Natural Language Processing with Python*. O'Reilly. [Practical implementation with NLTK]
- **Eisenstein, J. (2019).** *Introduction to Natural Language Processing*. MIT Press.

End of Class Notes — Unit 1: Introduction to NLP & Classical Approaches

Best of luck with your studies!